

LAPORAN TUGAS – FINAL PROJECT

Perancangan Analisis Algoritma – B

Dosen : Rully Soelaiman, S.Kom., M.Kom.

Maulana Ikhsan – 5025241163

Daftar Isi.....	
1. Masalah.....	1
a. Problem description	1
b. Problem analysis	2
2. Penyelesaian	3
a. Mathematical approach.....	3
b. Algorithmic approach	6
c. I/O and variable handling.....	10
3. Coding.....	12
a. Pseudocode	12
b. Complete program	17
4. Hasil	23
a. Correctness analysis.....	23
b. Complexity analysis.....	24
c. Submission proof	25

1. Masalah

a. Problem description

Pada final project ini diberikan soal dari platform SPOJ yaitu :

- Nama : NKDIST – Distance
- Link : <https://www.spoj.com/problems/NKDIST/>
- Ss :

NKDIST - Distance

no tags

[English](#) [Vietnamese](#)

Consider a sequence D consisting of an infinite number of hexadecimal digits made by concatenating all the positive integers 1, 2, 3, 4,..., N,...

The sequence D begins with:

123456789ABCDEF101112**13**1415161718191A1B1C1D1E1F20**21**22...

We may see D as an infinite string of hexadecimal digits. Let S be an arbitrary string consisting only of hexadecimal digits. The number of occurrences of S in D as a substring is infinite. The distance between two nonoverlapping occurrence of S is the number of digits between these two occurrences. For instance, if S='21', the distance between the first two occurrences of S is 27 (as illustrated above).

Task

You are given a string S of at most 30 characters long. Write a program that determines the distance between the first two occurrences of S in D.

Input

The input contains the string S in a single line.

Output

The output contains the distance between the first two occurrences of S in D in a single line.

Example

Input	21
Output	27
Input	A
Output	26

[Submit solution!](#)

- Deskripsi soal :
Consider a sequence D consisting of an infinite number of hexadecimal digits made by concatenating all the positive integers 1, 2, 3, 4,..., N,...

The sequence D begins with:

123456789ABCDEF101112**13**1415161718191A1B1C1D1E1F20**21**22...

We may see D as an infinite string of hexadecimal digits. Let S be an arbitrary string consisting only of hexadecimal digits. The number of occurrences of S in D as a substring is infinite. The distance between two nonoverlapping occurrence of S is the number of digits between these two occurrences. For instance, if S='21', the distance between the first two occurrences of S is 27 (as illustrated above).

Task :

You are given a string S of at most 30 characters long. Write a program that determines the distance between the first two occurrences of S in D.

Input : The input contains the string S in a single line.

Output : The output contains the distance between the first two occurrences of S in D in a single line.

Example :

Input

21

Output

27

Input

A

Output

26

b. Problem analysis

Masalah diatas menugaskan kita untuk mencari jarak antara kemunculan pertama string S dan kemunculan kedua string tersebut pada baris hexadecimal

$D = 123456789ABCDEF1011121314151617181.....$

Misal input S = "12", maka output dari masalah diatas adalah

$D = \text{12}3456789ABCDEF1011\text{12}1314151617181 \dots$

Dapat kita lihat bahwa jarak antara kemunculan pertama dan kedua adalah selisih dari jarak antara index terakhir kemunculan pertama dan index pertama kemunculan kedua yaitu

$Dist = D_1[n - 1] - D_2[0]$ atau bisa kita sebut sebagai jarak antara kemunculan pertama dan kedua, lalu ditambah panjang dari kemunculan pertama, amka :

$$Dist = D_1[n - 1] - D_2[0] \leftrightarrow D_1[0] - D_2[0] - n \leftrightarrow first - second - n$$

Berdasarkan constraint yang ada, diketahui bahwa $0 \leq len(s) \leq 30$. Maka dengan melakukan analisis batas, kita dapat menyimpulkan

- Lower bound : S = "1" $\rightarrow$ 14 char jarak first dan second
- Upper bound : S = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF" $\rightarrow$ estimasi kasar kemunculan pertama pada char ke 2×10^{37} dalam sequence D(penjelasan terkait hal ini akan dibahas pada bab 2.a).
- Maximum search : Jika kita memiliki input "S" dengan panjang m, maka paling maksimal kemunculan kedua dari "S" akan berada pada panjang m+1(worst case)
- Worst case ex : Ex. "FFF" $\rightarrow$ "1FFF"

Dengan begini, maka kita mengetahui terdapat 3 hal yang harus kita lakukan yaitu :

- Hitung kemunculan pertama string S pada sequence (first)
- Hitung kemunculan kedua string S pada sequence (second)
- Menyimpan nilai dalam variabel dengan kapasitas X, $X \geq 10^{38}$

2. Penyelesaian

a. Mathematical approach

Untuk mencari posisi angka N pada sequence, kita dapat menggunakan 2 approach:

1. Bruteforce

Untuk mencari ini, kita dapat generate semua kemungkinan string mulai dari 1 $\rightarrow$ infinity dan melacak posisinya menggunakan variabel sementara. Tetapi hal ini tidaklah efektif karena produksi string dapat mencapai angka yang sangat besar. Maka dari itu untuk ini dan selanjutnya kita dapat menyimpulkan menggunakan pendekatan matematis dengan metode divide and conquer

2. Mathematic generation

Untuk mencari ini, dengan melakukan pengibaratan menggunakan desimal kita mengetahui bahwa

- Terdapat 9 angka satuan pada bilangan asli $X \rightarrow X_1 \in [0,9]$
- Terdapat 99 angka puluhan, atau bisa kita sebut sebagai $|X_2| = 99 \rightarrow |X_2| = 90 + X_1 \rightarrow |X_2| = 10 \times X_1 + X_1$
- Jika sequence diatas kita panjangkan, maka akan menjadi seperti tabel

Labels	Digits	Total Numbers	Expression
X1	1	9	9×10^0
X2	2	99	$(9 \times 10^1) + (9 \times 10^0)$
X3	3	999	$(9 \times 10^2) + (9 \times 10^1) + (9 \times 10^0)$
...	...	...	
X(n)	n	...	$(9 \times 10^{n-1}) + (9 \times 10^{n-2}) + \dots + (9 \times 10^0)$

- Sehingga dapat dirumuskan, untuk sembarang bilangan dengan panjang k, terdapat $\sum_{n=1}^k 9 \times 10^{n-1}$ bilangan bulat
- Jika kita perhatikan, untuk setiap k digit, terdapat karakter sebanyak total sebanyak $K \times$ expression pada derajat k tersebut. Sehingga kita dapat menghitung jumlah karakter yang ada untuk semua bilangan dengan panjang K yaitu sebanyak

$$\sum_{n=1}^k n \times 9 \times 10^{n-1} \rightarrow 9 \times \sum_{n=1}^k n \times 10^{n-1}$$

Karakter untuk bilangan dengan basis 10.

- Hal yang serupa juga dapat kita lakukan untuk bilangan hexadecimal berbasis 16 dengan 15 buah digit satuan, kita hanya perlu mengganti pengali nya saja.

Berdasarkan soal, diketahui posisi string “S” akan berada pada posisi setelah S-1. Misalkan S merupakan string yang mengandung K digit, maka posisi pasti

kemunculan angka S pada string dapat dirumuskan sebagai jumlah semua karakter yang memiliki digit < digit(S)[N] ditambah dengan jumlah karakter yang kurang dari S dengan digit == digit(S)[X]. Maka $pos(S) = N + X$, maka

$$N = \sum_{x=1}^{k-1} X \times 15 \times 16^{X-1} \rightarrow$$

$$N = 15 \times \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow 15 \times M$$

$$M = \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow (1 \times 16^0) + (2 \times 16^1) + \dots + ((k-1) \times 16^{k-1})$$

$$16 \times M = 16 \times \sum_{x=1}^{k-1} X \times 16^{X-1} \rightarrow (1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)$$

$$16 \times M = \sum_{x=1}^{k-1} X \times 16^X \rightarrow (1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)$$

$$16M - M = [(1 \times 16^1) + (2 \times 16^2) + \dots + ((k-1) \times 16^k)] - [(1 \times 16^0) + (2 \times 16^1) + \dots + ((k-1) \times 16^{k-1})]$$

$$16M - M = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-2} + (k-1) \times 16^{k-1}$$

Ibaratkan G sebagai deret geometri pada 15M berisi $G = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-2}$

Maka, dengan

$A = -1$; $R = 16$. Nilai dari 15M dapat kita hitung sebagai jumlahan dari G dan juga $(k-1) \times 16^{k-1}$. Maka :

$$G = -1 \times 16^0 - 1 \times 16^1 - \dots - 1 \times 16^{k-1}$$

$$S_G = \frac{-1 \times (16^{k-1} - 1)}{(16 - 1)} = \frac{-16^{k-1} - 1}{15}$$

Maka,

$$15M = \frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1}$$

Berdasarkan perhitungan diatas, kita mendapatkan

$$N = 15M = \frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1}$$

Maka dengan ini kita sudah berhasil mencari untuk N. Lalu untuk X, diketahui banyaknya angka yang muncul sebelum string "S" dengan k digit adalah sebanyak $n - 16^{k-1}$. Maka banyaknya karakter yg muncul sebelum string "S" dengan K digit adalah

$$X = k \times (n - 16^{k-1})$$

Maka, posisi kemunculan angka "S" pada sequence terdapat pada index I dimana dalam bentuk closed form nya menjadi

$$I = \left(\frac{-16^{k-1} - 1}{15} + (k-1) \times 16^{k-1} \right) + (k \times (n - 16^{k-1}))$$

Atau dalam bentuk open form menjadi

$$I = \left(\sum_{x=1}^{k-1} X \times 15 \times 16^{x-1} \right) + (k \times (n - 16^{k-1}))$$

Berdasarkan analisis masalah pada 1.b, setelah mencari posisi absolut “S” maka kita akan mencari kemunculan pertama dari string S pada sequence. Terdapat 3 kemungkinan yang membuat string S muncul pada infinite sequence D, misal pada sequence S = “213” yaitu :

1. Muncul sebagai angka utuh
Ex. “...211212**213**21421...”

Kemunculan string “S” pada angka utuh terbagi menjadi beberapa kemungkinan yaitu :

- Nilai(S) secara utuh → Ex. “213”
Untuk ini maka value $n = \text{num}(S)$ tanpa offset(pergeseran bilangan apapun diawal index).
- Nilai(S) + random tail → Ex. “213A”
Untuk ini maka value $n = 16 \times \text{num}(S) + t, t \in [0, F]$ tanpa offset
- Random head + Nilai(S) → Ex. “A213”.
Untuk ini maka value $n = t \times 16^m + \text{num}(S), t \in [1, F]$ dengan 1 offset didepan.

2. Potongan parsial S pada n + potongan utuh pada n+1
Ex. “...10111**213**141...”

Kemunculan string “S” pada kasus ini dapat dianalisa dengan membagi S menjadi tail dan rest, misal input “213”. Maka tail = “**12**” dan rest = “**13**”. Maka untuk sembarang angka tail :

- $\text{Len}(S) = m$
- $\text{Len}(\text{tail}) = a$
- $\text{Len}(\text{rest}) = m - a$ (len input dikurangi yg sudah terpakai di tail)

Misalkan R merupakan sebuah bilangan utuh(bukan merupakan potongan substring angka lain dari $y = \text{tail} + 1$ dengan panjang L, maka jumlah kemungkinan himpunan $H(y)$ adalah

$H(y) = R[(0..l)-1]$ dengan offset didepan sepanjang $\text{len}(y) - a$ karakter → (maksudnya adalah offset akan terjadi mulai dari digit pertama pada angka y hingga digit ke-x pada angka y yang menjadi kemunculan pertama pada tail).

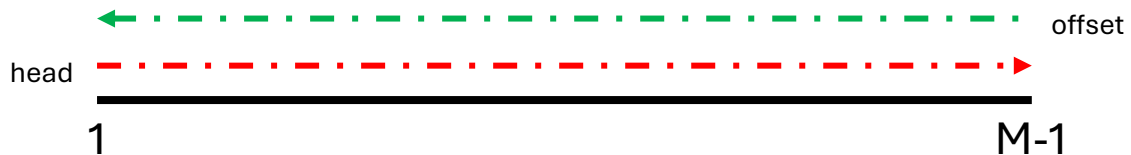
Maka kandidat hasil akan berada di $n = y - 1$ dengan offset = $\text{len}(n) - a$.

3. Potongan parsial S pada angka n dan n+1
Ex. “...3203**213**2232332...”

Kemunculan string “S” pada kasus ini dapat dianalisa dengan membagi S menjadi tail dan head(mirip seperti kasus2, tetapi tidak full memotong), misal input “213”. Maka tail = “3**21**” dan head = “**3**22”. Maka untuk tail :

- $\text{Len}(S) = m$
- $\text{Len}(\text{tail}) = a$
- $\text{Len}(\text{head}) = m - a$

Misalkan head merupakan prefix dari $y = n+1$ (sama seperti kasus2) dengan panjang L , maka y akan berada dalam range $S[1] \rightarrow S[m-1]$ (ini iterasi untuk mencari kemungkinan pembagian tail dan head) dengan offset sejauh $[m-1, 1]$.



Maka kandidat valid untuk kasus ini adalah y yang memenuhi syarat :

1. Syarat interval Y

Interval y berada pada

$$y \in [\min, \max] \rightarrow y \in [\text{left} \times 16^{l-\text{len}(\text{head})}, (\text{left} + 1) \times 16^{l-\text{len}(\text{head})} - 1]$$

note *misal $\min = \text{"EA****"}$, maka $\max = (\text{"EB0000"} - 1) = \text{"EAFFFF"}$

2. Syarat kongruensi nilai

Untuk setiap looping y pada interval yang ada akan dilakukan pengecekan nilai berdasarkan sifat kongruensi bilangan

- Karena head adalah prefix $y=n+1$, maka tail adalah suffix dari $y=n$
- Karena tail merupakan a digit terakhir dari n (lihat pada Ex., $a=2$) maka nilai desimal dari digit terakhir n adalah tail $\rightarrow n \bmod 16 \equiv \text{tail}$
- Maka, $n \bmod 16 \equiv \text{tail} \rightarrow y = n + 1 \rightarrow y \equiv (\text{tail} + 1) \bmod 16^a$

Jika kandidat angka y pada kasus ini memenuhi 2 syarat diatas, maka kita bisa menghitung semua kemungkinan kandidat valid y dengan (offset = $\text{len}(n) - a$) karakter dari depan.

Dengan melakukan hal diatas, maka kita telah berhasil mencari semua kemungkinan kemunculan S dan perhitungan posisi S dengan akurat dan efisien menggunakan approach Divide and Conquer.

b. Algorithmic approach

Algorithmic approach yang digunakan untuk problem ini adalah pendekatan dengan metode Divide and Conquer serta penyelesaian menggunakan approach Branch and Bound untuk semi-simulasi pruning pada kandidat jawaban yang ada dengan metode candidate enumeration seperti dibawah.

1. Overview algoritma

Algoritma untuk penyelesaian masalah ini bekerja secara 4 fase yaitu :

1. Precompute

Di fase ini dilakukan perhitungan untuk 16^x untuk mencegah misscalculation pada fungsi di algoritma dan juga menghemat waktu komputasi

2. String generation(candidate)

Di fase ini dilakukan analisis kasus kemungkinan valid string S pada sequence D dan perhitungan perkiraan substring valid berdasarkan perhitungan matematis pada 2.a

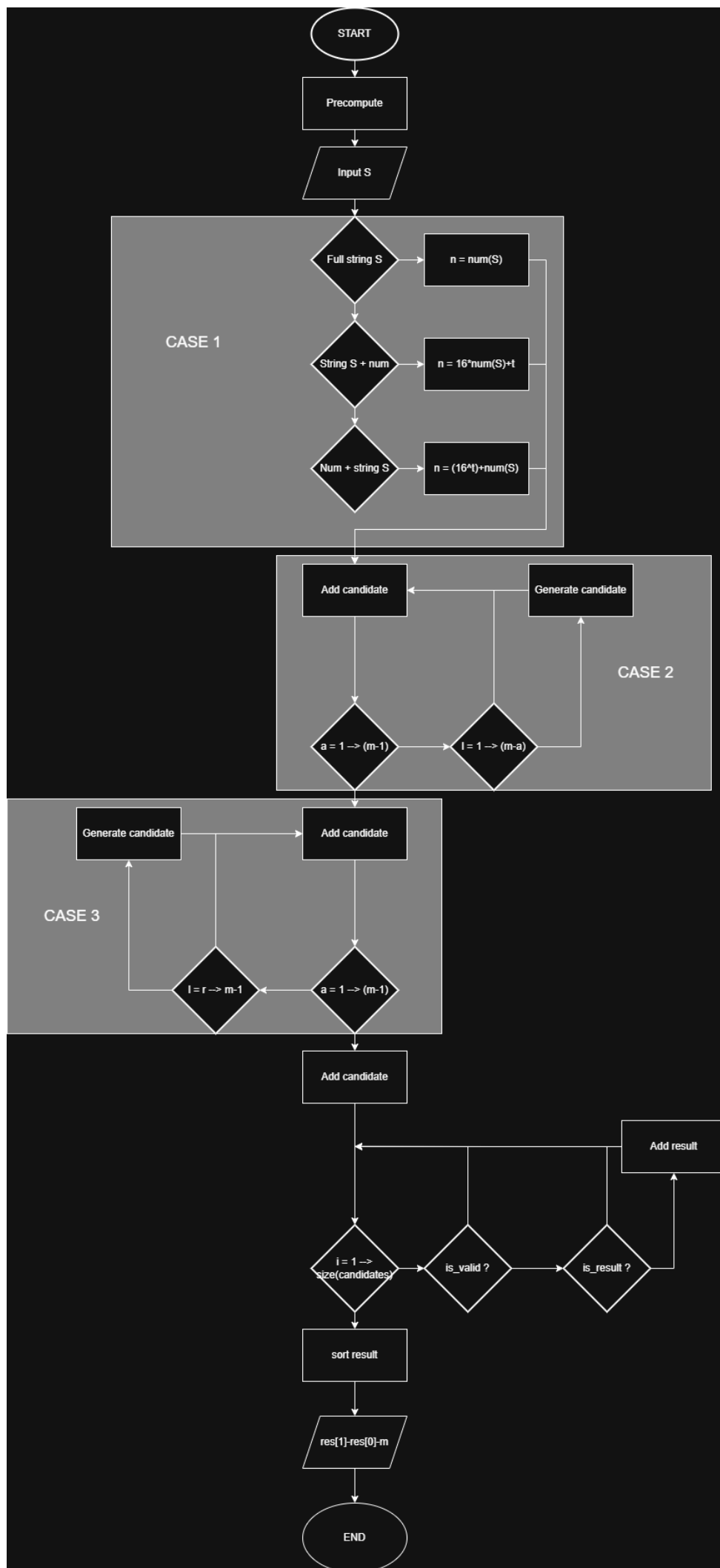
3. Seleksi jawaban

Di fase ini dilakukan perhitungan posisi pasti dari kemungkinan jawaban pada fase 2 dengan menggunakan offset sebagai acuan pergeseran substring valid dari kemungkinan penyusunan string pada sequence D

4. Kalkulasi jawaban untuk output

Di fase ini hanya menerapkan rumus jarak yang ada pada 1.b

Berikut merupakan flowchart lengkap dari algoritma untuk penyelesaian masalah pada bab1.



2. Penerapan OOP pada program

Pada program untuk penyelesaian problem ini, saya menggunakan approach OOP sebagai hasil akhir dari submission saya(sebelumnya saya menggunakan procedural programming pada first ac). Hal ini dilakukan dengan mempertimbangkan tingkat kesulitan soal serta kerumitan code yang pada akhirnya membuat program tampak tidak rapih serta tidak enak dipandang. Maka dari itu saya ubah penyelesaiannya dengan membagi beberapa operasi besar menjadi 3 class utama menggunakan konsep OOP.

Berikut merupakan penjelasan detail mengenai penggunaan 3 class utama serta penerapan konsep OOP pada struktur algoritma yang ada.

a. Class BigInt

Class ini dibuat untuk menampung elemen hasil yang tidak bisa ditampung dengan tipe data “unsigned int128” dari C++. Hal ini sudah terbukti dimana pada testcase S = “FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF” unsigned int128 memberikan output yang berbeda jika dibandingkan dengan perhitungan manual yang saya kerjakan. Class BigInt menerapkan konsep OOP dengan penggunaan

- Encapsulation

Digunakan enkapsulasi pada operasi trim0 yang akan diset secara default menggunakan public constructor serta pada operasi di public method dengan tujuan agar saya tidak harus menangani 0 overflow atau mismatch secara khusus.

- Abstraction

Digunakan abstraksi dengan operator overloading untuk mempermudah penggunaan operasi “+, -, <, >=, ==” sesuai dengan kebutuhan program agar penulisan kode dapat dilakukan secara rapih, terstruktur, serta lebih mudah untuk dibaca.

- Polymorphism

Digunakan static polymorphism pada constructor overload serta operator overload untuk mempermudah modelling pada program(walaupun sebenarnya tidak dibutuhkan).

b. Class Hexadecimal

Class ini dibuat untuk menjembatani operasi berbasis hexadecimal pada soal dengan perhitungan default komputer yang menggunakan basis 10(decimal). Pada dasarnya class ini tidak diperlukan jika kita treat operasi menggunakan binary, tetapi dikarenakan keterbatasan saya(saya tidak mampu untuk bikin seperti ini) maka saya mengimplementasikannya menggunakan class hexadecimal ini. Class hexadecimal menerapkan konsep OOP dengan penggunaan

- Abstraction

Digunakan abstraksi pada operasi konversi basis yang menjadi dasar dibuatnya class hexadecimal ini. Hal ini dilakukan untuk mengunci fungsi hexadecimal ini agar hanya bertindak sebagai konverter antar basis (walaupun sebenarnya ini bisa dilakukan hanya dengan namespace, tetapi tidak apa apa).

c. Class Algo

Class ini dibuat untuk implementasi logika matematis serta algoritma pada problem NKDIST ini. Class ini berperan untuk dekomposisi masing masing approach yang ada pada bab 2.a serta pada flowchart sebagai method tersendiri yang independen sehingga membuat public class menjadi bersih terhadap method overloading. Class Algo menerapkan konsep OOP dengan penggunaan

- Encapsulation

Digunakan enkapsulasi untuk mengunci semua private method yang ada agar nantinya public method hanya mengimplementasikan flow algoritma yang ada tanpa perlu memikirkan operasi lain yang berhubungan dengan perhitungan algoritma tersebut

- Abstraction

Digunakan abstraksi untuk mendekomposisi proses yang ada menjadi method independen yang membuat program dan algoritmanya menjadi mudah untuk debugging.

Sebenarnya secara umum hanya diperlukan 1 class yaitu BigInt untuk menyelesaikan problem NKDIST, tetapi saya menganggap approach saya acceptable karena dengan cara seperti ini akan mempermudah saya untuk debugging masing masing step yang ada (utamanya dikarenakan banyak sekali proses yang harus kita lakukan untuk solve problem ini).

c. I/O and variable handling

- Input

Input diambil dengan char temp[32] agar bisa dilakukan menggunakan scanf("%s", temp); lalu di parse menuju string S untuk perhitungan hasil operasi. Sebelumnya sudah sempat dicoba scanf langsung ke S tetapi menimbulkan error, maka dari itu hal ini dilakukan agar inputnya menjadi aman.

- Output

Output di-print dalam bentuk string hasil yang merupakan angka numerik sesuai dengan bab 1.b dengan perhitungan dalam bentuk class BigInt agar bisa diimplementasikan dengan mudah.

- Variable

Variabel yang ada dan berkaitan dengan program diurus menggunakan unsigned int128 pada C++ untuk menghemat memori yang ada. Tetapi dikarenakan terdapat kemungkinan overflow untuk kasus besar maka hasil akhir posisi index kandidat ke-i disimpan dalam class BigInt yang ada.

Digunakan juga komparator dengan struct mindec agar secara default kandidat jawaban yang ada pada class Algo disimpan secara sorted.

3. Coding

a. Pseudocode

Beirkut merupakan pseudocode parsial berisi rancangan masing masing class dan juga pseudocode akhir dari program yang saya bikin.

○ Class BigInt

```
CLASS BigInt
  DATA:
    value ← string

  PRIVATE FUNCTION TrimLeadingZeros(s)
    Find the first character in s that is not '0'
    IF no such character exists THEN
      RETURN "0"
    ELSE
      RETURN substring of s starting from the first non-
zero character

  CONSTRUCTOR BigInt()
    value ← "0"

  CONSTRUCTOR BigInt(s)
    value ← TrimLeadingZeros(s)

  CONSTRUCTOR BigInt(x)
    value ← ConvertUnsigned128ToString(x)

  FUNCTION ToString()
    RETURN value

  STATIC FUNCTION Compare(a, b)
    a ← TrimLeadingZeros(a)
    b ← TrimLeadingZeros(b)

    IF length of a < length of b THEN return -1
    IF length of a > length of b THEN return 1
    IF a = b THEN return 0
    IF a < b lexicographically THEN return -1
    ELSE return 1

  FUNCTION LessThan(other)
    RETURN Compare(value, other.value) < 0

  FUNCTION GreaterOrEqual(other)
    RETURN Compare(value, other.value) ≥ 0
```

```

FUNCTION Equal(other)
    RETURN Compare(value, other.value) = 0

FUNCTION Add(other)
    Perform digit-by-digit addition from right to left
    Handle carry
    Reverse the result
    RETURN a new BigInt from the result string

FUNCTION Subtract(other)
    Perform digit-by-digit subtraction from right to left
    Handle borrow
    Remove unnecessary trailing zeros in the reversed result
    Reverse the result
    RETURN a new BigInt from the result string

STATIC FUNCTION ConvertUnsigned128ToString(x)
    IF x = 0 THEN
        RETURN "0"
    Repeatedly take x MOD 10, store digits, and divide x by
10
    Reverse the collected digits
    RETURN the resulting string
END CLASS

```

- Class Hexadecimal

```

CLASS Hexadecimal

    STATIC FUNCTION HexToDec(s)
        result ← 0
        FOR each character c in s DO
            result ← result shifted left by 4 bits
            IF c is between '0' and '9' THEN
                result ← result + numeric value of c
            ELSE
                result ← result + hexadecimal value of c
        RETURN result

    STATIC FUNCTION DecToHex(num)
        IF num = 0 THEN
            RETURN "0"

        digits ← "0123456789ABCDEF"
        result ← empty string

        WHILE num > 0 DO
            digit ← num AND 15
            Prepend digits[digit] to result

```

```

        num ← num shifted right by 4 bits

    RETURN result

    STATIC FUNCTION HexLength(x)
        length ← 0
        REPEAT
            length ← length + 1
            x ← x shifted right by 4 bits
        UNTIL x = 0
        RETURN length
END CLASS

```

○ Class Algo

```

CLASS Algo
    DATA:
        inputnum ← unsigned 128-bit integer
        exp16 ← array of unsigned 128-bit integers
        input ← string
        m ← integer
        possible ← ordered set of strings using numeric
comparison

    PROCEDURE MemoizePowers()
        exp16[0] ← 1
        FOR i from 1 to 31 DO
            exp16[i] ← exp16[i - 1] × 16

    FUNCTION FoundFirst(n)
        k ← hexadecimal length of n
        pos ← BigInt(0)

        FOR d from 1 to k - 1 DO
            term ← d × 15 × 16^(d - 1)
            pos ← pos + term

        tail ← k × (n - 16^(k - 1))
        pos ← pos + tail

        RETURN pos

    FUNCTION FoundSecond(n, off)
        need ← off + length of input
        window ← empty string
        cur ← n

        WHILE length of window < need DO

```

```

        window ← window concatenated with hexadecimal
representation of cur
        cur ← cur + 1

    RETURN whether the substring of window starting at off
matches input

PROCEDURE AddResult(n, off)
    IF n = 0 THEN return
    IF hexadecimal length of n > m + 1 THEN return
    IF off < 0 OR off ≥ hexadecimal length of n THEN return
    IF FoundSecond(n, off) is false THEN return

    pos ← FoundFirst(n) + off
    Insert pos as a string into possible

PROCEDURE Case1()
    IF the first character of input is not '0' THEN
        AddResult(inputnum, 0)

        prefix ← inputnum shifted left by 4 bits
        FOR t from 0 to 15 DO
            AddResult(prefix + t, 0)

        FOR h from 1 to 15 DO
            n ← h × 16m + inputnum
            AddResult(n, 1)

PROCEDURE Case2()
    FOR a from 1 to m - 1 DO
        rest ← substring of input starting at a
        r ← m - a

        IF rest is empty OR first character of rest is '0'
THEN continue

        FOR l from 1 to r DO
            subfirst ← first l characters of rest
            IF first character of subfirst is '0' THEN
continue

            y ← hexadecimal value of subfirst
            IF hexadecimal length of y ≠ 1 THEN continue
            IF y = 0 THEN continue

            off ← hexadecimal length of (y - 1) - a
            AddResult(y - 1, off)

PROCEDURE Case3()

```

```

FOR a from 1 to m - 1 DO
    tail ← substring of input from 0 to a - 1
    head ← substring of input from a to end
    r ← m - a

    IF first character of head is '0' THEN continue

    mask ← 16^a - 1
    left ← hexadecimal value of head
    right ← (hexadecimal value of tail + 1) AND mask

    FOR l from r + 1 to m + 2 DO
        extra ← l - r
        IF extra < 1 OR extra ≥ 32 THEN continue

        block ← 16^extra
        low ← left × block
        high ← (left + 1) × block - 1
        IF high < low THEN continue

        rem ← low AND mask
        delta ← (right + 16^a - rem) AND mask
        y ← low + delta

        WHILE y ≤ high DO
            IF hexadecimal length of y = 1 THEN
                n ← y - 1
                off ← hexadecimal length of n - a
                AddResult(n, off)

            IF high - y < 16^a THEN break
            y ← y + 16^a

PROCEDURE Solve()
    Call MemoizePowers()

    Read input as a string
    inputnum ← hexadecimal value of input
    m ← length of input

    Call Case1()
    Call Case2()
    Call Case3()

    Convert possible set into an ordered array arr
    first ← arr[0]
    lim ← first + m

```

```

        Search for the first element in arr that is greater than
or equal to lim
        Store it as second

        result ← second - first - m
        Print result
END CLASS

```

Berikut merupakan final pseudocode dari program saya

```

BEGIN PROGRAM
    Create an object of class Algo
    Call the Solve procedure of the object
    Terminate the program
END PROGRAM

```

b. Complete program

Berikut merupakan full code saya terhadap problem ini

```

#include <algorithm>
#include <cctype>
#include <cstdio>
#include <iostream>
#include <set>
#include <string>
#include <vector>
using namespace std;
typedef unsigned __int128 u128;

class BigInt {
    string value;

    static string trim0(string s) {
        int p = (int)s.find_first_not_of('0');
        if (p == (int)string::npos) return "0";
        return s.substr(p);
    }

public:
    BigInt() : value("0") {}
    BigInt(string s) : value(trim0(s)) {}
    BigInt(u128 x) : value(u1282str(x)) {}
    string str() { return value; }

    static int compare(string a, string b) {
        string x = trim0(a), y = trim0(b);
        if (x.size() != y.size()) return x.size() < y.size() ? -
1 : 1;

```

```

        if (x == y) return 0;
        return x < y ? -1 : 1;
    }

    bool operator<(BigInt o) { return compare(value, o.value) <
0; }
    bool operator>=(BigInt o) { return compare(value, o.value)
>= 0; }
    bool operator==(BigInt o) { return compare(value, o.value)
== 0; }

    BigInt operator+(BigInt o) {
        string x = value, y = o.value;
        string out;
        int i = (int)x.size()-1, j = (int)y.size()-1, carry = 0;
        while (i >= 0 || j >= 0 || carry) {
            int da = (i >= 0 ? x[i] - '0' : 0);
            int db = (j >= 0 ? y[j] - '0' : 0);
            int sum = da + db + carry;
            out.push_back((char)('0' + (sum % 10)));
            carry = sum/10;
            --i; --j;
        }
        reverse(out.begin(), out.end());
        return BigInt(out);
    }

    BigInt operator-(BigInt o) {
        string x = value, y = o.value;
        string out;
        int i = (int)x.size()-1, j = (int)y.size()-1, borrow =
0;
        while (i >= 0) {
            int da = x[i] - '0' - borrow;
            int db = (j >= 0 ? y[j] - '0' : 0);
            if (da < db) {
                da += 10;
                borrow = 1;
            } else borrow = 0;
            out.push_back((char)('0' + (da - db)));
            --i; --j;
        }
        while (out.size() > 1 && out.back() == '0')
out.pop_back();
        reverse(out.begin(), out.end());
        return BigInt(out);
    }

    static string u1282str(u128 x) {

```

```

        if (x==0) return "0";
        string out;
        while (x > 0) {
            out.push_back((char)('0' + (int)(x % 10)));
            x/=10;
        }
        reverse(out.begin(), out.end());
        return out;
    }
};

struct mindec {
    bool operator()(string a, string b) const { return
BigInt::compare(a, b) < 0; }
};

class Hexadecimal {
public:
    static u128 hex2dec(string s) {
        u128 res = 0;
        for (char c : s) {
            res <<= 4;
            if (c >= '0' && c <= '9') res += (u128)(c - '0');
            else res += (u128)(c - 'A' + 10);
        }
        return res;
    }

    static string dec2hex(u128 num) {
        if (num == 0) return "0";
        string out, digits = "0123456789ABCDEF";
        while (num > 0) {
            out = digits[(int)(num & 15)] + out;
            num >>= 4;
        }
        return out;
    }

    static int hexlen(u128 x) {
        int len = 0;
        do {
            ++len;
            x >>= 4;
        } while (x > 0);
        return len;
    }
};

class Algo {

```

```

u128 inputnum, exp16[32]; //dp memoization
string input;
int m;
set<string, mindec> possible; //maybe ans

void memo() {
    exp16[0] = 1;
    for (int i = 1; i < 32; ++i) exp16[i] = exp16[i-1]<<4;
}

BigInt found1st(u128 n) { //hasil nurunin
    int k = Hexadecimal::hexlen(n);
    BigInt pos;
    for (int d = 1; d < k; ++d) {
        u128 term = (u128)d * (u128)15 * exp16[d-1];
        pos = pos+BigInt(term);
    }
    u128 tail = (u128)k * (n - exp16[k-1]);
    pos = pos+BigInt(tail);
    return pos;
}

bool found2nd(u128 n, int off) { //offset dikit
    int need = off + (int)input.size();
    string window;
    u128 cur = n;
    while ((int>window.size()) < need) {
        window += Hexadecimal::dec2hex(cur);
        ++cur;
    }
    return window.compare(off, input.size(), input) == 0;
}

void add_res(u128 n, int off) {
    if (n == 0) return;
    if (Hexadecimal::hexlen(n) > m + 1) return;
    if (off < 0 || off >= Hexadecimal::hexlen(n)) return;
    if (!found2nd(n, off)) return;
    BigInt pos = found1st(n)+BigInt((u128)off);
    possible.insert(pos.str());
}

void case1() {
    if (input[0] != '0') {
        add_res(inputnum, 0);
        u128 prefix = inputnum << 4;
        for (int t = 0; t < 16; ++t) add_res(prefix + (u128)t,
0);
    }
}

```

```

        for (int h = 1; h <= 15; ++h) {
            u128 n = (u128)h * exp16[m] + inputnum;
            add_res(n, 1);
        }
    }

    void case2() {
        for (int a = 1; a < m; ++a) {
            string rest = input.substr(a);
            int r = m-a;
            if (rest.empty() || rest[0] == '0') continue;

            for (int l = 1; l <= r; ++l) {
                string subfirst = rest.substr(0, l);
                if (subfirst[0] == '0') continue;

                u128 y = Hexadecimal::hex2dec(subfirst); //y = n+1
                if (Hexadecimal::hexlen(y) != l) continue;
                if (y == 0) continue;

                int off = Hexadecimal::hexlen(y - 1) - a;
                add_res(y-1, off);
            }
        }

        void case3() {
            for (int a = 1; a < m; ++a) {
                string tail = input.substr(0, a), head =
input.substr(a);
                int r = m-a;
                if (head[0] == '0') continue;

                u128 mask = exp16[a] - 1;
                u128 left = Hexadecimal::hex2dec(head), right =
(Hexadecimal::hex2dec(tail)+1) & mask;

                for (int l = r + 1; l <= m + 2; ++l) {
                    int extra = l-r;
                    if (extra < 1 || extra >= 32) continue;

                    u128 block = exp16[extra];
                    u128 low = left*block, high = (left + 1)*block -
1;

                    if (high < low) continue;

                    u128 rem = low & mask;
                    u128 delta = (right + exp16[a] - rem) & mask;
                    u128 y = low + delta;

```

```

        while (y <= high) {
            if (Hexadecimal::hexlen(y) == 1) {
                u128 n = y - 1;
                int offside = Hexadecimal::hexlen(n) -
a;

                add_res(n, offside);
            }
            if (high-y < exp16[a]) break;
            y += exp16[a];
        }
    }
}

public:
    void solve() {
        memo();
        char temp[32];
        scanf("%s", temp);

        input = temp;
        inputnum = Hexadecimal::hex2dec(input);
        m = (int)input.size();

        case1();
        case2();
        case3();

        vector<string> arr(possible.begin(), possible.end());
        string first = arr[0], second;
        string lim = (BigInt(first) + BigInt((u128)m)).str();
        bool found = false;

        for (int i = 1; i < (int)arr.size(); ++i) {
            if (BigInt::compare(arr[i], lim) >= 0) {
                second = arr[i];
                found = true;
                break;
            }
        }

        string result = (BigInt(second)-BigInt(first)-
BigInt(to_string(m))).str();
        printf("%s\n", result.c_str());
    }
};

int main() {
    Algo fp;

```

```
fp.solve();  
return 0;  
}
```

4. Hasil

a. Correctness analysis

Untuk membuktikan kebenaran program saya, dilakukan 5 metode yaitu

1. Penetapan aturan dasar

- D merupakan infinite string yang dibentuk dari penggabungan bilangan natural hexadecimal berurutan tanpa leading 0.
 $D = 123456789ABCD\dots$
- S merupakan sembarang string hexadecimal dengan panjang m karakter dimana $m \leq 30$
- Kemunculan S dicatat dengan index awal kemunculan S pada D
- Jika kemunculan pertama S adalah P_1 dan kemunculan kedua S adalah P_2 , maka jawaban atas pertanyaan yang dibutuhkan adalah $P_2 - P_1 - m$

2. Pernyataan proposisi

“Untuk setiap input string S, algoritma akan mengembalikan jarak tepat diantara 2 kemunculan S pertama yang tidak bersinggungan”.

Untuk membuktikan ini, terdapat 3 hal untuk dibuktikan yaitu :

- Setiap kemungkinan kandidat yang dihasilkan algoritma merupakan kemunculan valid dari S
- Setiap kemunculan S dihasilkan dari salah satu kasus pada algoritma
- Program menghitung kemunculan valid pertama dan kedua lalu menghitung jaraknya secara akurat

3. Pemilihan strategi pembuktian

Digunakan “proof by exhaustion” untuk membuktikan kemunculan kandidat jawaban dan “direct proof” untuk membuktikan rumus evaluasi jarak. Dengan 3 lema(teorema pembantu) yang harus dibuktikan yaitu :

- Lema A; jumlah digit sebelum kemunculan S
- Lema B; setiap kemunculan kandidat jawaban harus memenuhi kasus pada algoritma
- Lema C; rumus jarak antara 2 kemunculan pertama

4. Eksekusi logika

- Lema A

Untuk sembarang angka N hexadecimal dengan panjang k , jumlah digit sebelum N terbagi mejadi 2 panjng dengan rumus

1. $\text{Len} < k$

$$\sum_{d=1}^{k-1} d \times 15 \times 16^{d-1}$$

2. $\text{Len} == k$

$$k \times (n - 16^{k-1})$$

maka terdapat x buah digit dimana

$$X = \sum_{d=1}^{k-1} d \times 15 \times 16^{d-1} + k \times (n - 16^{k-1})$$

Terbukti

○ Lema B

Sebuah string S pada infinite hexadecimal string D, pasti muncul dengan salah diantara 2 kasus yaitu

- Muncul sebagai angka utuh
- Muncul sebagai angka parsial

Dengan semua variannya, terdapat total 4 kemungkinan yaitu

1. "...[S]..."
2. "...[S+'c']..."
3. "...['c'+S]..."
4. "...[...+T][H+...]..." → semua kemungkinan variasinya juga termasuk

Terbukti

○ Lema C

Misal terdapat sebuah string S dengan panjang m yang muncul pada index X_1 dan X_2 di D dan kita asumsikan X_1 dan X_2 tidak bersinggungan. Maka

- Index awal X_1 = posisi X_1
- Index akhir X_1 = posisi $X_1 + m$
- Index awal X_2 = posisi X_2
- Index akhir X_2 = posisi $X_2 + m$

Selisih = index awal x_2 – index akhir x_1

Selisih = $X_2 - (X_1 + m) \rightarrow X_2 - X_1 - m$

Terbukti

5. Verifikasi

Dikarenakan semua poin pada no.2 sudah terbukti, maka dapat dipastikan bahwa algoritma pada soal ini terbukti kebenarannya.

b. Complexity analysis

Berikut merupakan analisa kompleksitas dari program saya untuk permasalahan NKDIST ini

○ Time complexity

Berikut merupakan time complexity untuk tiap komponen dengan operasi didalamnya. Dikarenakan limitasi pada input membuat $m \leq 30$, maka untuk worst case terdapat $30^3 = 27.000$ operasi yang dilakukan, tetap tergolong cepat dan efisien.

function	complexity	detail
memo()	$O(1)$	memoization total 32x loop
case1()	$O(m)$ $O(m^3)$	iterasi string murni valid
case2()		build up kandidat dari kombinasi potongan string
case3()	$O(m^3)$	build up kandidat dari kombinasi potongan string
result	$O(m^2)$	search kemungkinan kombinasi jawaban

○ Memory complexity

Memory complexity untuk program ini akan berjumlah sesuai maksimum set possible yang digenerate oleh tiap kandidat. Untuk setiap set yang menyimpan $O(m^2)$ string BigInt(didapat dari case generate). Maka kompleksitas memori untuk program ini adalah sebesar $O(m^2)$.

c. Submission proof

Berikut merupakan bukti submission dari problem ini



sphere online judge

 PROBLEMS
  STATUS
  RANKS
  DISCUSS
  CONTESTS
  PROFILE

Problems

/

Distance

/

My submissions


 Not hidden submissions
 All submissions

Ikhsan_163: submissions

Distance

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35693580	 2026-04-20 17:39:30	Distance	100 edit ideone.it	0.07	5.2M	CPP
35693544	 2026-04-20 17:33:23	Distance	0 edit ideone.it	0	0k	CPP
35691446	 2026-04-20 00:41:42	Distance	100 edit ideone.it	0.06	5.2M	CPP
35691371	 2026-04-19 23:45:34	Distance	100 edit ideone.it	0.08	5.2M	CPP
35691365	 2026-04-19 23:42:24	Distance	100 edit ideone.it	0.07	5.2M	CPP
35691358	 2026-04-19 23:41:20	Distance	0 edit ideone.it	0	0k	CPP
35691354	 2026-04-19 23:40:13	Distance	0 edit ideone.it	0	0k	CPP
35691290	 2026-04-19 23:05:22	Distance	100 edit ideone.it	0.06	5.2M	CPP
35691289	 2026-04-19 23:04:26	Distance	0 edit ideone.it	0	0k	CPP
35690883	 2026-04-19 18:40:54	Distance	90 edit ideone.it	0.05	5.2M	CPP
35690881	 2026-04-19 18:40:13	Distance	compilation error edit ideone.it	-	-	CPP